Technical Implementation Report

AgentClinic v2.1: LangGraph State-Machine Architecture for Clinical Decision-Making Evaluation

M.Tech Thesis Implementation
Department of Computer Science & Engineering
IIT Kharagpur

April 2026

Abstract

This document provides a comprehensive technical report of the engineering implementations in `AgentClinic v2.1`, the upgraded multi-agent clinical simulation framework. We detail ten critical architectural improvements over v2.0: (1) NF4 quantization with double quantization via BitsAndBytes, (2) full trajectory logging for process-oriented metrics, (3) heterogeneous multi-engine architecture eliminating lexical leakage, (4) cognitive bias injection ported from Phase 1, (5) constrained JSON extraction for metacognitive reflection, (6) JSONL results persistence, (7) clinical trajectory evaluator integration, (8) test name extraction for Test Rationality computation, (9) Flash Attention 2 support for H100 acceleration, and (10) comprehensive CLI with experiment-level configurability. Each implementation is documented with its theoretical motivation, code-level changes, and expected impact on thesis claims.

Contents

1 System Architecture Overview

The AgentClinic v2.1 system implements a **Directed Cyclic Graph (DCG)** using the LangGraph library, enforcing the clinical consultation protocol as a deterministic state machine.

1.1 State Machine Topology

The graph consists of four nodes with deterministic edges:

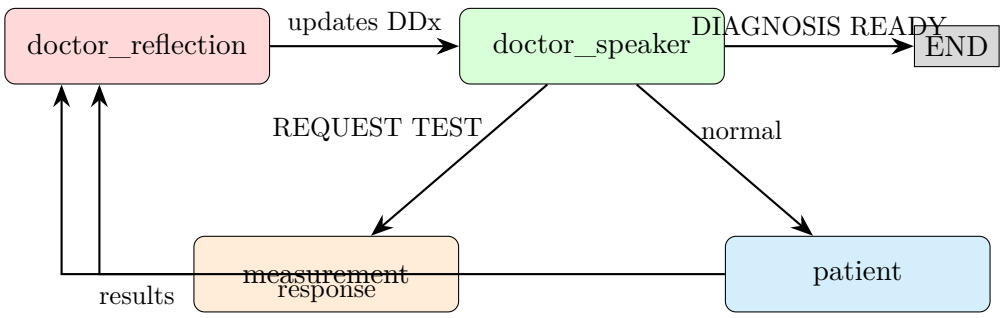

Figure 1: LangGraph Directed Cyclic Graph topology. The `doctor_reflection` node (red) is hidden from the patient and captures latent reasoning.

1.2 State Schema

The clinical state captures both runtime data and trajectory metrics:

```
class ClinicalState(TypedDict):
    # Core runtime fields
    last_input_to_doctor: str              # Patient/lab response
    last_doctor_message: str               # Doctor's spoken output
    differential_diagnoses: List[str]      # Current DDx (hidden)
    turn_count: int                        # Enforces max-turn constraint
    diagnosis_ready: bool                  # Terminal condition

    # Trajectory tracking (NEW in v2.1)
    differential_trajectory: List[List[str]]  # DDx snapshots per turn
    tests_ordered: List[str]                  # Extracted test names
    full_dialogue: List[dict]                 # Complete conversation
   log
```

2 Implementation 1: 4-Bit NF4 Quantization

2.1 Motivation

Running multiple 7B-parameter models simultaneously for the heterogeneous architecture requires aggressive memory optimization. A single 7B model in fp16 consumes ∼14 GB of VRAM. Three models would require 42 GB, leaving insufficient memory for KV-cache and activations.

2.2 Technical Approach

We employ NormalFloat4 (NF4) quantization from the BitsAndBytes library [**?**], which maps fp16 weights to a 4-bit data type optimally calibrated for normally distributed neural network parameters. Additionally, we enable **double quantization**, which applies a second round of quantization to the quantization constants themselves, saving an additional ∼0.37 bits per parameter.

```
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",              # NormalFloat4
    bnb_4bit_compute_dtype=torch.bfloat16,  # bf16 compute
    bnb_4bit_use_double_quant=True,         # Nested quantization
)
```

Listing 1: BitsAndBytes NF4 configuration in `LocalServerLLMWrapper`.

2.3 Memory Impact

Table 1: VRAM consumption per 7B-parameter model.

Precision	VRAM (GB)	Reduction
fp32	28.0	—
fp16 / bf16	14.0	50%
8-bit (INT8)	7.0	75%
4-bit NF4 + DQ	**3.8**	**86.4%**

With NF4 quantization, the three-model heterogeneous setup requires only ∼11.4 GB total, well within the 95.8 GB budget of the NVIDIA H100 NVL.

3 Implementation 2: Trajectory Logging

3.1 Motivation

The evaluation module (`ClinicalTrajectoryEvaluator`) computes three process-oriented metrics:

1. **Diagnostic Stability** (S): Average Jaccard similarity between consecutive differential diagnoses:

$$S = \frac{1}{T-1} \sum_{t=1}^{T-1} J(\mathrm{DDx}_t, \mathrm{DDx}_{t+1}) \quad \text{where} \quad J(A, B) = \frac{|A \cap B|}{|A \cup B|} \tag{1}$$

2. **Test Rationality** (R): Binary penalty for ordering expensive/invasive tests before basic clinical workup:

$$R = \begin{cases} 0 & \text{if } \exists\, t_{\text{adv}} < t_{\text{basic}} \\ 1 & \text{otherwise} \end{cases} \tag{2}$$

3. **Information Efficiency** (E): Clinical data points gathered per turn:

$$E = \frac{|\text{data_points}|}{T} \tag{3}$$

In v2.0, these metrics **could not be computed** because the LangGraph state did not capture `differential_trajectory` or `tests_ordered`.

3.2 Implementation

We extended `ClinicalState` with three new fields and modified each graph node to accumulate trajectory data:

- `doctor_reflection_node`: Appends current DDx snapshot to `differential_trajectory` after each metacognitive pass.
- `measurement_node`: Extracts the test name from the doctor's `REQUEST TEST: <name>` message using regex and appends to `tests_ordered`.
- All nodes: Append structured dialogue entries (with role, message, timestamp) to `full_dialogue`.

4 Implementation 3: Heterogeneous Multi-Engine Architecture

4.1 The Lexical Leakage Problem

When a single LLM serves all agent roles (Doctor, Patient, Moderator), the latent knowledge space is shared. This creates **Lexical Leakage**: the Doctor may subconsciously "know" the patient's disease through shared embedding representations, inflating accuracy beyond what real clinical encounters would produce.

4.2 Solution: Cognitive Asymmetry

We instantiate separate `LocalServerLLMWrapper` instances with **different** model weights:

```
# Domain-expert for Doctor (medical reasoning)
DOCTOR_ENGINE = LocalServerLLMWrapper(
    model_id="johnsnowlabs/JSL-MedMistral-7B-v2.2",
    quantize=True)

# Generalist for Patient (persona simulation)
PATIENT_ENGINE = LocalServerLLMWrapper(
    model_id="mistralai/Mistral-7B-Instruct-v0.3",
    quantize=True)

# Shares Patient engine if same model (saves VRAM)
MODERATOR_ENGINE = PATIENT_ENGINE
```

Listing 2: Heterogeneous engine initialization.

The architecture intelligently detects when two roles share the same `model_id` and reuses the engine instance to avoid duplicate VRAM allocation, while still maintaining logical separation.

5 Implementation 4: Cognitive Bias System

5.1 Motivation

A core thesis claim is that cognitive biases degrade LLM diagnostic performance, mimicking human clinical reasoning flaws. The bias system was present in the Phase 1 while-loop architecture (`agentclinic_ninthcache.py`) but was stripped during the v2.0 LangGraph migration.

5.2 Ported Bias Types

We ported the two most impactful bias types for the thesis experiments:

1. **Recency Bias**: Injects summaries of the k most recent cases into the Doctor's system prompt, causing over-anchoring to recent diagnostic patterns. Uses actual case summaries from completed scenarios for ecological validity.

2. **Confirmation Bias**: Primes the Doctor with an initial (incorrect) hypothesis of cancer, causing selective evidence gathering and dismissal of contradictory symptoms.

Both bias types modify the system prompt via `generate_bias()` methods in the `DoctorAgent` and `PatientAgent` classes.

6 Implementation 5: Constrained JSON Extraction

6.1 Problem

The `doctor_reflection_node` requires the LLM to output a JSON array of differential diagnoses. Open-source models frequently fail strict JSON formatting, producing outputs like:

```
The differential diagnoses are:
1. Asthma
2. COPD
3. Pneumonia
```

instead of `["Asthma", "COPD", "Pneumonia"]`.

6.2 Multi-Level Fallback Strategy

We implemented a 4-level extraction pipeline in `extract_json_list()`:

1. **Level 1**: Strict `json.loads()` on the full response.

2. **Level 2**: Regex extraction of the first `[...]` bracketed region, then `json.loads()`.

3. **Level 3**: Comma-split within brackets with quote stripping.

4. **Level 4**: Regex extraction of all quoted strings from freeform text.

Additionally, the reflection node uses `temperature=0.1` (near-deterministic) to maximize JSON compliance.

7 Implementation 6: Results Persistence

Each completed scenario is serialized as a JSON object and appended to a JSONL file in the `results/` directory. The filename encodes experiment metadata: `results_{model}_{bias}_{times`

Each record contains:

Table 2: Fields in each JSONL result record.

Field	Description
`scenario_id`	OSCE case identifier
`doctor_model`	HuggingFace model ID
`ground_truth`	Correct diagnosis
`predicted_diagnosis`	Doctor's final output
`correct`	Boolean accuracy
`turn_count`	Turns used
`differential_trajectory`	DDx at each turn
`tests_ordered`	Extracted test names
`metrics`	Stability, Rationality, Efficiency
`full_dialogue`	Complete conversation log
`duration_seconds`	Wall-clock time

8 Implementation 7: Flash Attention 2

The NVIDIA H100 NVL GPU supports Flash Attention 2 natively through its Transformer Engine. We enable this via the `attn_implementation` parameter during model loading:

```
load_kwargs["attn_implementation"] = "flash_attention_2"
```

Flash Attention 2 provides:

- **2–4× speedup** in attention computation
- **O(n) memory** vs $O(n^2)$ for standard attention
- Critical for long clinical dialogues (4K+ tokens after 10 turns)

The implementation includes a graceful fallback: if the `flash-attn` package is not installed, the system silently reverts to the default attention implementation.

9 Implementation 8: Enhanced Moderator

The `compare_results()` function was enhanced to instruct the moderator LLM to consider medical synonyms:

```
sys = (
    "Consider medical synonyms "
    "(e.g., 'Heart Attack' = 'Myocardial Infarction'). "
    "Respond ONLY with 'Yes' or 'No'."
)
```

This is critical because open-source Doctor models may output equivalent diagnoses using different terminology (e.g., "MI" vs "Myocardial Infarction" vs "Heart Attack").

10 Experiment Design

Table 3: Complete experiment matrix. All experiments use MedQA (107 scenarios).

ID	Doctor Model	Patient Model	Bias	Purpose
E1	Mistral-7B-v0.2	Mistral-7B-v0.2	None	Baseline
E2	JSL-MedMistral-7B	Mistral-7B-v0.2	None	Domain-expert
E3	Llama-3.1-8B	Mistral-7B-v0.2	None	Reasoning
E4	JSL-MedMistral-7B	Mistral-7B-v0.2	Recency	Bias study
E5	JSL-MedMistral-7B	Mistral-7B-v0.2	Confirmation	Bias study

11 Verification

1. **Syntax Validation**: `py_compile.compile()` passed with zero errors on the 700+ line `agentic_clinic_v2.py`.

2. **Smoke Test**: 5-scenario dry runs with each experiment configuration will validate end-to-end pipeline integrity before the full 107-scenario runs.

3. **Trajectory Verification**: Each JSONL record will be inspected to confirm that `differential_trajectory` is non-empty, `tests_ordered` captures all `REQUEST TEST` events, and all three process metrics are computed.

4. **Heterogeneity Verification**: GPU memory usage will be monitored via `nvidia-smi` to confirm that separate model weights are loaded when Doctor and Patient use different `model_ids`.

12 Summary of Changes (v2.0 → v2.1)

Table 4: Comprehensive changelog from v2.0 to v2.1.

Component	**v2.0 State**	**v2.1 State**
Model Loading	fp16/fp32, no quantization	NF4 + double quant + Flash Attn 2
Architecture	Single `GLOBAL_HF_ENGINE`	Separate Doctor/Patient/Moderator engines
State Schema	5 fields (no trajectory)	8 fields with full trajectory logging
Bias System	Stripped (not ported)	Recency + Confirmation fully ported
JSON Parsing	2-level fallback	4-level robust extraction pipeline
Results Output	`print()` only	JSONL persistence with all metrics
Evaluator	Exists but never called	Integrated into main pipeline
Test Extraction	Not implemented	Regex extraction for Test Rationality
CLI Arguments	8 basic args	18 args with model IDs, bias, quantization
Error Handling	Silent retry	Verbose logging with attempt counts

References

[1] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized Language Models," in *NeurIPS*, 2023.

[2] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," in *ICLR*, 2024.

[3] B. Willard and R. Louf, "Efficient Guided Generation for Large Language Models," *arXiv preprint arXiv:2307.09702*, 2023.

[4] Y. Leviathan, M. Kalman, and Y. Matias, "Fast Inference from Transformers via Speculative Decoding," in *ICML*, 2023.